Instituto Tecnológico de Costa Rica

Proyecto #1

Estudiante:

Quiriat Mata Araya – 2023379891

Elder Perez Leon - 2023166120

Profesor:

Allan Rodríguez Dávila

Curso:

Compiladores e Interpretes

G60

Semestre I 2026

Manual de Usuario 3
Estructura del proyecto: 3
Copilación: 3
Ejecución: 3
Flujo de ejecución: 3
Archivos de prueba: 4
Pruebas de funcionalidad 4
Archivo valido: 4
Archivo Invalido: 6
Descripcion del problema 7
Diseño del programa 8
Arquitectura general 8
Main (Main.java) 8
Lexer (Lexer.flex) 9
Parser (Parser.cup) 9
TablaSimbolos.java 10
Librerias usadas: 10
Objetivos alcanzados 11
Decisiones de Diseño: 11
Repositorio GitHub: 12

Manual de Usuario

Estructura del proyecto:

```
Proyecto/
    lib/
        jflex.java
        java-cup.jar
        java-cup-runtime.jar
    src/
        app/Main.java
        lexer/Lexer.flex
        parser/
            Parser.cup
            TablaSimbolos.java
    EjemplosPruebas/
        archivo1.txt
        archivo2.txt
```

Copilación:

Abra una terminal en la raíz del proyecto (carpeta Proyecto/) y ejecute:

- javac -cp ".;lib/jflex.jar;lib/java-cup-runtime.jar" src/app/Main.java

Ejecución:

Una vez compilado, ejecute el siguiente comando desde la raíz del proyecto:

- java -cp ".;lib/jflex.jar;lib/java-cup-runtime.jar;src/app" Main

Flujo de ejecución:

Al ejecutar Main.java el programa realiza los siguientes pasos de forma autónoma:

- Elimina archivos Java generados previamente: Lexer.java, MyParser.java, sym.java.
- Genera Lexer.java desde src/lexer/Lexer.flex usando JFlex.
- Genera MyParser.java y sym.java desde src/parser/MyParser.cup usando CUP.

- Compila en orden: sym → TablaSimbolos → Lexer → MyParser.
- Para cada archivo .txt en EjemplosPruebas/ (excepto los que empiecen con tokens_):
- Limpia la tabla de símbolos.
- Ejecuta el análisis léxico y genera tokens_<nombre>.txt.
- Ejecuta el análisis sintáctico (que llena la tabla de símbolos).
- Imprime la tabla de símbolos, el resumen de errores léxicos y sintácticos, y el veredicto final.

Archivos de prueba:

Para probar el proyecto coloque archivos con extensión .txt en la carpeta EjemplosPruebas/. El sistema los procesará automáticamente en la siguiente ejecución. Los archivos de salida de tokens se guardan como tokens_<nombre_original>.txt en la misma carpeta.

Pruebas de funcionalidad

Archivo valido:

Ejemplo.txt:

```
string~_f_1<|bool~param1_|>|:return~fl1^p1/'3'-
"@#764~!:?/><"!:|float~_f_2<||>|:int~miArr<<34>><<23>>!int~miArr2<<34>><<23>><-
|:|:1,3:|,|:4,5:|,|:4,5:|:|!miArr2<<34*f<|param1_@true,equal<|op1,op2|>,1,id,0.5|>>><<
23-3>><-4!otro<-miArr2<<34*f<|param1_@true,equal<|op1,op2|>,1,id,0.5|>>><<23-
3>>+5!int~var1<-34/f<|param1_@true,equal<|op1,op2|>,1,id,0.5|>!return~var1-        -
1!:|float~_f_2<|bool~param1_,bool~param1_,bool~param1_|>|:string~s!bool~b!char~
c!string~c<-
45!do|:if<|34/f<|param1_@true,equal<|op1,op2|>,1,id,0.5|>|>|:float~id!float~id!break!
:|bool~b2!if<|34/f<|param1_@true,equal<|op1,op2|>,1,id,0.5|>|>|:float~id!float~id!bre
ak!if<|34/f<|param1_@true,equal<|op1,op2|>,1,id,0.5|>|>|:float~id!float~id!break!swit
ch<|34/f<|param1_@true,equal<|op1,op2|>,1,id,0.5|>|>|:case~3:case~'3':break!case
~

2:hola<-3!default:hola<-3!:|switch<|34/f|>|:case~3:default:hola<-
3!:|switch<|34/f|>|:case~3:hola<-
3!break!:|:|:|else|:float~id!float~id!break!:|:|while<|op1^34e56|>!string~s2!:|empty~__
main__<||>|:return~f<|op1^34e56+2//45,equal<|op1+-0.5,op2|>@op2#$id|>!:|
```

Salida de consola:

```
═══════════════════════════════
  Archivo: Ejemplo.txt
═══════════════════════════════

[ LÉXICO ]
──────────────────────────────────────────────────────

? Sin errores léxicos.

[ TABLA DE SÍMBOLOS ]

Scope: [ global ]
──────────────────────────────────────────────────────
TOKEN               TIPO              FILA      COLUMNA
──────────────────────────────────────────────────────
_f_1                FUNCION           1         8
_f_2                FUNCION           1         71
_f_2                FUNCION           1         377

Scope: [ _f_1 ]
──────────────────────────────────────────────────────
TOKEN               TIPO              FILA      COLUMNA
──────────────────────────────────────────────────────
param1_             PARAMETRO         1         19

Scope: [ _f_2 ]
──────────────────────────────────────────────────────
TOKEN               TIPO              FILA      COLUMNA
──────────────────────────────────────────────────────
miArr               ARREGLO           1         85
miArr2              ARREGLO           1         107
var1                VARIABLE          1         300
```

```
Scope: [ _f_22 ]
──────────────────────────────────────────────────────
TOKEN               TIPO              FILA      COLUMNA
──────────────────────────────────────────────────────
param1_             PARAMETRO         1         388
param1_             PARAMETRO         1         401
param1_             PARAMETRO         1         414
s                   VARIABLE          1         432
b                   VARIABLE          1         439
c                   VARIABLE          1         446
c                   VARIABLE          1         455
s2                  VARIABLE          2         173

Scope: [ do_while__f_22 ]
──────────────────────────────────────────────────────
TOKEN               TIPO              FILA      COLUMNA
──────────────────────────────────────────────────────
b2                  VARIABLE          1         550

Scope: [ if_do_while__f_22 ]
──────────────────────────────────────────────────────
TOKEN               TIPO              FILA      COLUMNA
──────────────────────────────────────────────────────
id                  VARIABLE          1         525
id                  VARIABLE          1         534

Scope: [ if2_do_while__f_22 ]
──────────────────────────────────────────────────────
TOKEN               TIPO              FILA      COLUMNA
──────────────────────────────────────────────────────
id                  VARIABLE          1         613
id                  VARIABLE          1         622
```

```
Scope: [ if_if2_do_while__f_22 ]
TOKEN                TIPO               FILA       COLUMNA
id                   VARIABLE           1          691
id                   VARIABLE           1          700

Scope: [ switch_if_if2_do_while__f_22 ]
TOKEN                TIPO               FILA       COLUMNA
(sin identificadores)

Scope: [ switch2_if_if2_do_while__f_22 ]
TOKEN                TIPO               FILA       COLUMNA
(sin identificadores)

Scope: [ switch3_if_if2_do_while__f_22 ]
TOKEN                TIPO               FILA       COLUMNA
(sin identificadores)

Scope: [ else_do_while__f_22 ]
TOKEN                TIPO               FILA       COLUMNA
id                   VARIABLE           2          125
id                   VARIABLE           2          134
```

```
Scope: [ __main__ ]
TOKEN                TIPO               FILA       COLUMNA
(sin identificadores)

[ PARSER ]
? Sin errores sintácticos.
 El archivo cumple con la gramática y puede ser procesado.
```

Archivo Invalido:

Ejemplo.txt:

empty ~ __main__ <| |> |:

  int ~ x <- 5 ¡¡ falta fin de sentencia “!”

  if <| x |> |:

    cout <| x |>!

    int ~ x <- 5!

¡¡ falta “:|” al final

Salida en consola:

```
═══════════════════════════════════
  Archivo: Ejemplo.txt
═══════════════════════════════════

[ LÉXICO ]
──────────────────────────────────────────────────────────
? Sin errores léxicos.
Error sintáctico en línea 3: token inesperado 'if'
Error sintáctico en línea -1: token inesperado 'desconocido'
Error sintáctico irrecuperable en línea -1. No se pudo continuar el análisis.

[ TABLA DE SÍMBOLOS ]

Scope: [ __main__ ]
──────────────────────────────────────────────────────────
TOKEN                TIPO               FILA       COLUMNA
──────────────────────────────────────────────────────────
x                    VARIABLE           5          11
```

```
[ PARSER ]
──────────────────────────────────────────────────────────
? Errores sintácticos encontrados: 4
  Error sintáctico en línea 3: token inesperado 'if'
    ? Recuperado: se omitió sentencia inválida hasta '!'
  Error sintáctico en línea -1: token inesperado 'desconocido'
  Error sintáctico irrecuperable en línea -1. No se pudo continuar el análisis.
──────────────────────────────────────────────────────────
El archivo NO cumple con la gramática. Total de errores: 4
═══════════════════════════════════
```

Cuando el parser reporta un error en línea -1, significa que un error sintáctico se arrastró hasta el final del archivo sin poder ser resuelto. Esto ocurre porque el parser seguía esperando un token para completar una estructura (por ejemplo, un cierre de bloque :|) pero en lugar de recibirlo, encontró el EOF (fin de archivo), que es la señal de que ya no hay más código que leer.

Como el EOF no es un token real del archivo sino una señal generada automáticamente al terminar la lectura, no tiene una posición de línea asociada, por lo que su línea se reporta como -1.

La decisión de diseño fue conservar este comportamiento intencionalmente, ya que un valor de -1 en la línea es una señal clara y reconocible de que el archivo terminó de forma incompleta. Esto puede deberse a un bloque |: sin su cierre :|, una expresión sin terminar, o una sentencia sin su ! final en la última línea del archivo.

Por eso en este ejemplo se reporta correctamente el error en la línea 3, pues encuentra un token inesperado en esa línea (if), y así podemos saber que probablemente esté faltando el cierre de sentencia (!)en la línea anterior, pero en el cierre de bloque faltante “:|” reporta error en línea -1 por que ese error se arrastra hasta el final de archivo.

Descripcion del problema

El proyecto consiste en comprender la fase del analisis léxico y análisis sintactico de un compilador para la gramática desarrollada en la tarea 1 del curso. Esta gramatica utiliza una sintaxis no convencional con respecto a los lenguajes más comunes (C, Java, Python), con el objetivo de ilustrar que los conceptos del compilador son independientes de la notación.

Algunas características de la gramática son:

- Comentarios de línea con ¡¡ y de bloque con {- ... -}
- Operadores de comparación expresados con palabras clave: equal, n_equal, less_t, etc.
- Bloques delimitados con |: y :| en lugar de llaves { }
- Paréntesis representados con <| y |>
- Asignación con <- en vez de =
- Fin de sentencia con ! en lugar de ;
- Separador de nombre de variable ~ (por ejemplo: int ~ x)
- Tipos de datos: int, float, bool, string, char
- Literales numéricos especiales: fracción (a//b), exponencial (ae b)

El objetivo del proyecto es construir un analizador léxico (Lexer) y un analizador sintáctico (Parser) que sean capaces de reconocer programas válidos en este lenguaje, reportar errores con recuperación, y sentar las bases para etapas posteriores del compilador (tabla de símbolos, semántica, generación de código).

Diseño del programa

Arquitectura general

El sistema sigue una arquitectura clásica de compilador en pipeline:

```
Archivo .txt
  ↓
 Lexer (JFlex)      → Tokens + erroresLexicos[]
  ↓
 MyParser (CUP)     → Validación + erroresSintacticos[] + TablaSimbolos
  ↓
 Main.java          → Orquestación, salida y reporte
```

Main (Main.java)

Main.java es el punto de entrada del sistema. En la versión actualizada se refactorizaron las responsabilidades en métodos privados con nombres descriptivos, mejorando la legibilidad respecto a la versión anterior:

- limpiarArchivosAnteriores(): elimina los .java generados previamente.
- generarLexer() / generarParser(): invocan JFlex y CUP respectivamente.
- compilarProyecto(): compila en el orden correcto de dependencias (sym → TablaSimbolos → Lexer → MyParser).

- procesarArchivo(): coordina el análisis léxico, sintáctico y la presentación de resultados para cada archivo.
- Usa reflexión (URLClassLoader) para cargar dinámicamente las clases generadas sin recompilar Main.

Lexer (Lexer.flex)

El lexer reconoce todos los tokens del lenguaje mediante expresiones regulares. Las decisiones clave son:

- Orden de reglas por especificidad: palabras clave antes que ID, y literales numéricos de mayor a menor especificidad (EXPONENCIAL > FLOTANTE > FRACCION > ENTERO) para evitar tokenizaciones incorrectas.
- La regla FRACCION usa // como separador, declarada después de FLOTANTE y antes del operador DIVISION_ENTERA // para que el contexto (dígito//dígito) se tokenice como FRACCION y no como dos DIVISIONes.
- La regla catch-all [^] captura cualquier carácter no reconocido, lo agrega a erroresLexicos (lista estática) y continúa sin lanzar excepción, implementando recuperación léxica completa.
- Los campos left y right del Symbol almacenan línea y columna (yyline+1, yycolumn+1), que la tabla de símbolos usa como fila y columna.

Parser (Parser.cup)

generada con CUP. Cambios y decisiones clave:

- La tabla de símbolos ahora se puebla desde las acciones semánticas del parser. Cada producción que declara un identificador (funcion, param, declaracion_int, declaracion_float, sentencia) llama a TablaSimbolos.agregar().
- Los scopes se abren y cierran explícitamente: entrarScope() antes del bloque y salirScope() después. Esto se aplica a funcion, main, if_sentencia, else_bloque, do_while y switch_sentencia.
- Las llamadas a función (llamada_funcion) se registran con tipo LLAMADA pero TablaSimbolos.agregar() las filtra y no las inserta, evitando contaminación de la tabla.
- Declaracion_int y declaracion_float ahora distinguen entre VARIABLE (con expresión o sin inicializador) y ARREGLO (con dimensiones <<ENTERO>><<ENTERO>>).
- La recuperación de errores sintácticos se mantiene en la producción sentencia con error FIN_SENTENCIA, sincronizando en el token “!”.

TablaSimbolos.java

Estructura de datos para el manejo de scopes anidados. Componentes principales:

- tablas: LinkedHashMap<String, List<Simbolo>> que preserva el orden de inserción de los scopes.
- scopes: Deque<String> que funciona como pila; el tope es el scope activo.
- contadores: Map<String, Integer> para numerar scopes con el mismo nombre (ej.: dos if en la misma función generan if___main__ e if2___main__).
- El nombre de cada scope se construye como: nombre_padre (ej.: if___main__, do_while___main__). Si ya existe un scope con esa clave, el contador lo convierte en nombre2_padre. Esto garantiza unicidad sin pérdida de información jerárquica.

El método agregar() ignora silenciosamente las entradas de tipo LLAMADA, ya que las llamadas a función no constituyen declaraciones de identificadores.

Librerias usadas:

Libreria	**Proposito**
JFlex	Generador de analizadores léxicos. Convierte Lexer.flex en Lexer.java
CUP	Generador de parsers LALR(1). Convierte MyParser.cup en MyParser.java y sym.java
java-cup-runtime	Runtime requerido para ejecutar los parsers generados por CUP (Symbol, etc.)
java.lang.reflect	URLClassLoader, Class.forName, Field, Method: carga y acceso dinámico a clases generadas
java.util.LinkedHashMap	Almacena la tabla de símbolos preservando el orden de inserción de scopes
java.util.ArrayDeque	Pila de scopes activos (Deque) para entrarScope/salirScope
java.util.HashMap	Contador de nombres de scope repetidos para generar claves únicas
java.io.FileReader / PrintWriter	Lectura de archivos de entrada y escritura de tokens_*.txt
java.net.URLClassLoader	Carga dinámica del Lexer, MyParser y TablaSimbolos compilados

Objetivos alcanzados

- Generación automática del Lexer y del Parser desde archivos de especificación (.flex y .cup) integrada en el flujo de ejecución.
- Reconocimiento correcto de todos los tokens: palabras clave, operadores simbólicos y de palabra, literales entero/flotante/exponencial/fracción, cadenas, caracteres e identificadores.
- Recuperación léxica completa: caracteres no reconocidos se reportan sin detener el análisis.
- Recuperación sintáctica: errores se sincronizan con FIN_SENTENCIA (!), permitiendo continuar con el resto del archivo.
- Tabla de símbolos funcional con scopes anidados, distinguiendo VARIABLE, ARREGLO, PARAMETRO y FUNCION.
- Numeración automática de scopes repetidos (if, else, do_while, switch) dentro del mismo padre.
- Salida de tokens por archivo en tokens_<nombre>.txt.
- Veredicto final automático con conteo total de errores.
- Refactorización de Main.java con métodos bien separados y comentados.

Decisiones de Diseño:

Asociatividad mediante recursividad por la derecha con 'resto'

Para lograr asociatividad izquierda sin recursividad izquierda directa (que generaría conflictos en LALR(1)) se usa el patrón 'resto':

```
expresion_aritmetica ::= termino resto_aritmetica
resto_aritmetica     ::= SUMA   termino resto_aritmetica
                       | RESTA  termino resto_aritmetica
                       | /* vacio */
```

Este patrón implementa correctamente la evaluación de izquierda a derecha. Se aplica de igual forma a termino/resto_termino (*, /, %, //) y potencia/resto_potencia (^).

Operadores relacionales con notación funcional

Los operadores equal, n_equal, less_t, less_te, greather_t, greather_te se expresan como llamadas funcionales: equal <| a , b |>. Esta decisión elimina la ambigüedad de precedencia entre el nivel relacional y el aritmético, ya que los operandos se delimitan explícitamente con paréntesis. No se necesitan reglas adicionales de precedencia para este nivel.

Scopes con nombre jerárquico y contador de repetidos

Cada scope recibe el nombre nombre_padre (ej.: if___main__). Cuando el mismo tipo de estructura aparece más de una vez en el mismo scope padre, un contador genera nombres como if2___main__, if3___main__, etc. Esto permite:

- Identificar unívocamente cualquier bloque en la tabla de símbolos.
- Conservar la relación jerárquica en el nombre, facilitando la depuración.
- Evitar colisiones sin necesidad de UUIDs u otras estructuras complejas.

Filtrado de LLAMADA en la tabla de símbolos

Las llamadas a función (llamada_funcion) disparan un agregar() con tipo LLAMADA, pero TablaSimbolos.agregar() las descarta silenciosamente. Esta decisión mantiene la tabla limpia (solo declaraciones).

Repositorio GitHub:

https://github.com/Elder19/Proyecto1Compiladores-S12026.githttps://github.com/Elder19/Proyecto1Compiladores-S12026.git